ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN



ĐỀ XUẤT ĐỒ ÁN CUỐI KỲ

Môn: Vật lý cho Công nghệ thông tin

Phòng học thông minh

Giám sát chất lượng không khí và cảnh báo nhiệt độ, độ ẩm

(Smart Classroom: Air Quality Monitoring and Temperature–Humidity Alert)

Giảng viên hướng dẫn: Bùi Thanh Lâm
Cao Xuân Nam
Đặng Hoài Thương

Sinh viên thực hiện: Nguyễn Gia Quốc Uy – 24127261
Ngô Quốc Tuấn – 24127581
Nguyễn Quốc Thành – 24127542

Năm học: 2026 – 2027

TP. Hồ Chí Minh, Tháng 10 năm 2026

Mục lục

1 Giới thiệu đề tài **1**

2 Phạm vi chức năng **2**

3 Kiến trúc hệ thống **3**
3.1 Sơ đồ tổng thể 3
3.2 Các luồng xử lý chi tiết 3
3.3 Ngưỡng mặc định 5

4 Cài đặt cho người dùng không chuyên **5**

5 Công nghệ sử dụng **6**
5.1 Thiết bị phần cứng 6
5.2 Phương pháp ước lượng CO_2 từ MQ-135 7
5.3 Công nghệ phần mềm 8

6 Chi phí dự kiến **9**

1. Giới thiệu đề tài

Phòng học thường đóng kín và đông người. Khi thông gió kém, nồng độ CO_2 tăng nhanh gây buồn ngủ, giảm tập trung, nhưng người trong phòng không tự nhận biết được. Nhiệt độ quá cao ảnh hưởng sức khỏe và làm thiết bị điện quá nhiệt; độ ẩm quá cao gây đọng hơi nước, độ ẩm quá thấp gây tĩnh điện.

Nhóm đề xuất một thiết bị IoT đặt trong phòng học, gồm board ESP32 đọc cảm biến nhiệt độ, độ ẩm (DHT22) và CO_2 ước lượng (MQ-135), gửi dữ liệu qua MQTT đến website do nhóm tự xây dựng. Website hiển thị số liệu theo thời gian thực và biểu đồ lịch sử, điều khiển đèn LED RGB báo mức chất lượng không khí, điều khiển còi cảnh báo khi nhiệt độ, độ ẩm ở mức nguy hiểm, và gửi thông báo Telegram. Sản phẩm được làm thật, đóng hộp, và người dùng không có kiến thức công nghệ thông tin vẫn tự cài đặt được qua giao diện.

- **Nhóm:** Thành (DHT22), Uy (MQ-135), Tuấn (LED RGB, còi); mỗi người phụ trách một luồng căn bản với thiết bị riêng.
- **Hình thức:** sản phẩm thật, đóng hộp.
- **Yêu cầu nâng cao dự kiến thực hiện:** #3, #4, #5, #6, #9, #10, #12.
- **Chi phí linh kiện dự kiến:** khoảng 480.000đ.

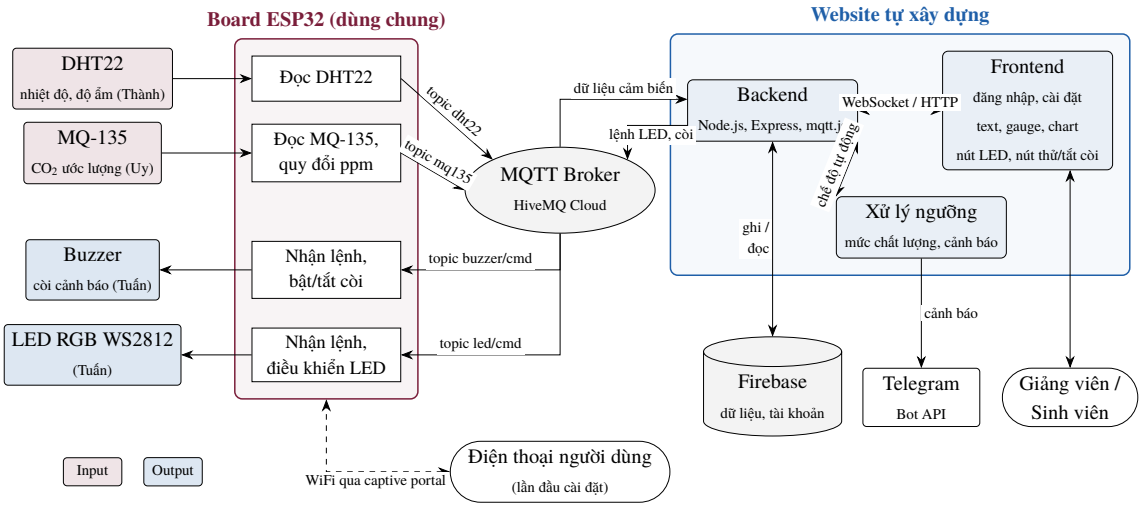
2. Phạm vi chức năng

Mã	Use case	Yêu cầu tương ứng	Phụ trách
UC1	Xem nhiệt độ, độ ẩm theo thời gian thực	Căn bản, luồng (1)	Thành
UC2	Xem CO_2 ước lượng theo thời gian thực	Căn bản, luồng (1)	Uy
UC3	Điều khiển LED RGB từ web: tự động theo mức chất lượng không khí hoặc chọn màu thủ công	Căn bản, luồng (2)	Tuấn
UC4	Còi cảnh báo khi nhiệt độ, độ ẩm vượt ngưỡng; thử và tắt còi từ web	Nâng cao #3	Tuấn
UC5	Lưu dữ liệu cảm biến theo thời gian lên cloud	Nâng cao #4	Uy
UC6	Xem biểu đồ lịch sử	Nâng cao #5	Thành
UC7	Nhận thông báo Telegram khi chất lượng không khí xấu	Nâng cao #6	Tuấn
UC8	Đăng ký, đăng nhập; tài khoản lưu trong database	Nâng cao #9	Thành
UC9	Cấu hình WiFi qua giao diện khi cài đặt lần đầu	Nâng cao #12	Uy
UC10	Xem mức chất lượng không khí (Tốt / Trung bình / Kém)	Phục vụ UC3, UC7	Tuấn
UC11	Chỉnh ngưỡng cảnh báo trên trang Cài đặt	Phục vụ UC4, UC7, UC10	Uy

Website do nhóm tự xây dựng, không dùng NodeRED (yêu cầu nâng cao #10).

3. Kiến trúc hệ thống

3.1 Sơ đồ tổng thể



Hình 1: Sơ đồ hoạt động của hệ thống.

3.2 Các luồng xử lý chi tiết

Luồng 1 — Nhiệt độ, độ ẩm lên website (UC1, Thành, căn bản luồng 1)

1. DHT22 đo nhiệt độ và độ ẩm trong phòng.
2. ESP32 đọc DHT22 vài giây một lần, đóng gói thành bản tin, ví dụ `{"temp":28.5,"hum":62}`.
3. ESP32 gửi (publish) bản tin qua WiFi lên topic `dht22` tại HiveMQ Cloud.
4. HiveMQ Cloud chuyển bản tin đến backend, vì backend đã đăng ký topic `dht22`.
5. Backend kiểm tra dữ liệu hợp lệ và gắn thời điểm nhận.
6. Backend đẩy số liệu đến trình duyệt qua WebSocket; frontend cập nhật ô text và gauge ngay, người dùng không cần tải lại trang.

Luồng 2 — CO_2 lên website (UC2, Uy, căn bản luồng 1)

1. MQ-135 thay đổi điện trở theo nồng độ khí, tạo ra một điện áp tương tự.
2. ESP32 đọc điện áp qua ADC, tính điện trở cảm biến và quy đổi ra nồng độ CO_2 ước lượng (ppm) theo phương pháp ở mục 5.2.
3. ESP32 publish bản tin lên topic `mq135` tại HiveMQ Cloud.
4. HiveMQ Cloud chuyển bản tin đến backend.
5. Backend kiểm tra dữ liệu hợp lệ và gắn thời điểm nhận.
6. Backend đẩy số liệu đến frontend qua WebSocket; frontend cập nhật ô text và gauge.

Luồng 3 — Lưu trữ và biểu đồ lịch sử (UC5 của Uy, UC6 của Thành)

1. Mỗi khi nhận bản tin ở luồng 1 và luồng 2, backend đồng thời ghi giá trị kèm thời điểm nhận vào Firebase.
2. Người dùng mở trang biểu đồ và chọn khoảng thời gian (theo giờ hoặc theo ngày).
3. Frontend gửi yêu cầu HTTP đến backend.
4. Backend truy vấn Firebase theo khoảng thời gian đã chọn và trả kết quả.
5. Frontend vẽ biểu đồ nhiệt độ, độ ẩm, CO_2.

Luồng 4 — Điều khiển LED RGB (UC3, Tuấn, căn bản luồng 2)

1. Người dùng chọn màu trên website, hoặc chuyển LED sang chế độ tự động.
2. Frontend gửi lệnh đến backend.
3. Backend kiểm tra người dùng đã đăng nhập. Ở chế độ tự động, backend quy đổi giá trị CO_2 mới nhất ra mức chất lượng (UC10) theo ngưỡng ở mục 3.3 để chọn màu: xanh lá (Tốt), vàng (Trung bình), đỏ (Kém); mức chất lượng cũng được hiển thị trên website.
4. Backend publish lệnh lên topic `led/cmd` tại HiveMQ Cloud.
5. HiveMQ Cloud chuyển lệnh đến ESP32, vì ESP32 đã đăng ký topic `led/cmd`.
6. ESP32 điều khiển LED WS2812 hiển thị màu tương ứng.

Ở chế độ tự động, bước 3 đến bước 6 tự lặp lại mỗi khi luồng 2 làm mức chất lượng thay đổi.

Luồng 5 — Còi cảnh báo (UC4, Tuấn, nâng cao #3)

1. *Thủ công:* người dùng bấm nút thử còi hoặc tắt còi; frontend gửi lệnh đến backend.
2. *Tự động:* mỗi khi luồng 1 mang số liệu mới, backend so nhiệt độ, độ ẩm với ngưỡng (mục 3.3); nếu vượt ngưỡng, backend tạo lệnh bật còi.
3. Backend publish lệnh lên topic `buzzer/cmd`, ví dụ `{"state":"on"}`.
4. HiveMQ Cloud chuyển lệnh đến ESP32.
5. ESP32 bật hoặc tắt còi. Khi số liệu trở lại bình thường hoặc người dùng bấm tắt, backend gửi lệnh `{"state":"off"}`.

Luồng 6 — Thông báo Telegram (UC7, Tuấn, nâng cao #6)

1. *Liên kết một lần:* người dùng bấm nút "Kết nối Telegram", ứng dụng Telegram mở ra, bấm Start; bot báo cho backend mã cuộc trò chuyện và backend lưu mã này vào Firebase.
2. *Khi cảnh báo:* nếu CO_2 ở mức Kém liên tục quá thời gian quy định (mục 3.3), backend gọi Telegram Bot API.
3. Telegram gửi tin nhắn cảnh báo đến điện thoại người dùng.

Luồng 7 — Đăng ký, đăng nhập (UC8, Thành, nâng cao #9)

1. Người dùng nhập tài khoản trên frontend; frontend gửi đến backend.
2. Backend đối chiếu với tài khoản lưu trong Firebase (khi đăng ký thì tạo mới).
3. Nếu hợp lệ, backend cấp phiên đăng nhập; mọi thao tác xem và điều khiển trên website đều phải kèm phiên này.
4. Lần đầu đăng nhập, người dùng nhập mã thiết bị để gắn thiết bị với tài khoản; backend lưu liên kết này vào Firebase.

Luồng 8 — Cấu hình WiFi lần đầu (UC9, Uy, nâng cao #12)

1. Khi chưa có thông tin WiFi, ESP32 tự phát một mạng WiFi riêng.
2. Người dùng kết nối bằng điện thoại; trang cấu hình tự hiện ra.
3. Người dùng chọn WiFi nhà, nhập mật khẩu; ESP32 lưu vào bộ nhớ flash và khởi động lại.
4. ESP32 kết nối WiFi, kết nối HiveMQ Cloud, rồi bắt đầu luồng 1 và luồng 2.

3.3 Ngưỡng mặc định

Hệ thống có sẵn bộ ngưỡng dưới đây; người dùng có thể chỉnh lại trên trang Cài đặt (UC11).

Đại lượng	**Ngưỡng mặc định**	**Phản hồi của hệ thống**
CO_2 ước lượng	< 800 ppm	LED xanh lá (Tốt)
	800–1200 ppm	LED vàng (Trung bình)
	> 1200 ppm	LED đỏ (Kém)
	> 1200 ppm liên tục 10 phút	Gửi thông báo Telegram
Nhiệt độ	$> 35\,^{\circ}\mathrm{C}$	Bật còi
Độ ẩm	$> 85\%$ hoặc $< 30\%$	Bật còi

Ràng buộc thiết bị đôi một khác nhau: DHT22 (Thành), MQ-135 (Uy), LED RGB (Tuấn) và Buzzer (Tuấn) mỗi thiết bị chỉ thuộc một luồng. Các yêu cầu nâng cao còn lại không dùng thêm thiết bị phần cứng.

4. Cài đặt cho người dùng không chuyên

Các cấu hình kỹ thuật (cloud, MQTT broker, Telegram bot) do nhóm thiết lập sẵn. Người dùng chỉ thao tác trên giao diện:

1. **Kết nối WiFi (UC9):** lần đầu cắm điện, thiết bị phát một mạng WiFi riêng; người dùng kết nối bằng điện thoại, trang cấu hình tự hiện ra để chọn WiFi và nhập mật khẩu.
2. **Tạo tài khoản (UC8):** đăng ký trên website và gắn thiết bị vào tài khoản.
3. **Kết nối Telegram (UC7):** bấm một nút trên website, sau đó bấm **Start** trong Telegram.
4. **Tùy chỉnh (UC11, không bắt buộc):** kéo thanh trượt để đổi ngưỡng cảnh báo; hệ thống có sẵn ngưỡng mặc định.

5. Công nghệ sử dụng

5.1 Thiết bị phần cứng

Thiết bị	**Vai trò và cách kết nối**	**Nguyên lý vật lý**
ESP32 DevKit	Vi điều khiển trung tâm, lõi kép 240 MHz, có WiFi 2,4 GHz và ADC 12-bit; chạy firmware, kết nối WiFi và MQTT	Truyền dữ liệu bằng sóng điện từ 2,4 GHz; ADC lấy mẫu điện áp tương tự và chuyển thành số, đo tuyến tính tốt trong khoảng 0,15–2,45 V
DHT22	Đo nhiệt độ (-40 đến $80\,^{\circ}$C) và độ ẩm (0–100%); giao tiếp số qua 1 dây	Độ ẩm làm thay đổi hằng số điện môi của lớp polymer trong tụ điện, nên điện dung thay đổi; nhiệt độ làm thay đổi điện trở của nhiệt điện trở
MQ-135	Đo chất lượng không khí, quy đổi ước lượng sang CO_2 tương đương; ngõ ra tương tự, qua cầu chia áp vào chân ADC	Lớp bán dẫn SnO_2 được nung nóng hấp phụ oxy trên bề mặt; khí khử (NH_3, hơi cồn, benzen, khói) phản ứng với oxy này, trả electron về lớp bán dẫn và làm điện trở giảm. Cảm biến không chọn lọc riêng CO_2, nên giá trị CO_2 chỉ là ước lượng
LED RGB WS2812	Báo mức chất lượng không khí bằng màu; giao tiếp số 1 dây	Ba LED đỏ, lục, lam phát quang khi electron tái hợp với lỗ trống ở tiếp giáp p-n; màu cuối là tổng hợp cộng màu, độ sáng từng kênh điều chỉnh bằng PWM
Buzzer	Phát âm thanh cảnh báo; điều khiển qua chân GPIO và transistor	Hiệu ứng áp điện nghịch: điện áp biến thiên làm tấm gốm biến dạng, dao động và phát ra sóng âm

5.2 Phương pháp ước lượng CO_2 từ MQ-135

MQ-135 không đo trực tiếp nồng độ CO_2; nhóm ước lượng qua bốn bước, nhằm cho giá trị ổn định và nhất quán chứ không đặt mục tiêu độ chính xác tuyệt đối.

Bước 1 — Từ điện áp ra điện trở cảm biến. Cảm biến (điện trở R_S) mắc nối tiếp với điện trở tải R_L trên module, cấp nguồn $V_C = 5$ V. Điện áp ngõ ra (tối đa 5 V) được đưa qua cầu chia áp, với R_1 nối tiếp và R_2 nối đất, sao cho điện áp vào ADC không vượt khoảng 2,5 V, nằm trong vùng đo tuyến tính của ADC ESP32:

$$V_{\text{out}} = V_{\text{ADC}} \frac{R_1 + R_2}{R_2}, \qquad R_S = R_L \frac{V_C - V_{\text{out}}}{V_{\text{out}}}.$$

Bước 2 — Hiệu chuẩn R_0. Sau khi làm nóng cảm biến 24–48 giờ, đặt thiết bị ở ngoài trời thoáng, nơi nồng độ CO_2 xấp xỉ mức nền khí quyển $C_0 \approx 420$ ppm, đo R_S và tính điện trở chuẩn

$$R_0 = R_S \left(\frac{C_0}{a} \right)^{1/b}.$$

R_0 được lưu vào bộ nhớ flash của ESP32 và dùng cho các lần đo sau.

Bước 3 — Quy đổi theo đường đặc tuyến. Đường đặc tuyến của nhà sản xuất cho CO_2 có dạng hàm lũy thừa:

$$C_{\text{CO}_2} = a \left(\frac{R_S}{R_0} \right)^{-b}, \qquad a \approx 116{,}6,\ b \approx 2{,}769.$$

Bước 4 — Lọc nhiễu. Mỗi lần đo lấy trung bình nhiều mẫu ADC liên tiếp, đọc điện áp đã hiệu chỉnh của ESP32 (ADC của ESP32 không tuyến tính ở hai đầu thang đo), sau đó lấy trung bình trượt theo thời gian để loại các dao động ngắn.

Giới hạn. Vì SnO_2 phản ứng với nhiều loại khí khử, sự xuất hiện của hơi cồn, khói hoặc NH_3 sẽ làm giá trị ước lượng tăng theo. Vì vậy hệ thống dùng giá trị này để phân mức chất lượng không khí (Tốt / Trung bình / Kém) thay vì công bố như một phép đo CO_2 chính xác.

5.3 Công nghệ phần mềm

Thành phần	Công nghệ	Vai trò
Firmware	C++ trên Arduino framework, PlatformIO; thư viện DHT, Adafruit NeoPixel, PubSubClient, ArduinoJson	Đọc cảm biến, điều khiển LED và còi, gửi/nhận bản tin MQTT
Cấu hình WiFi	Thư viện WiFiManager	Phát WiFi riêng và trang cấu hình khi cài đặt lần đầu
Giao thức	MQTT (publish/subscribe) qua TLS	Truyền dữ liệu giữa ESP32 và backend theo yêu cầu đồ án
MQTT broker	HiveMQ Cloud (gói miễn phí)	Trung chuyển bản tin; thiết bị và backend kết nối ra ngoài nên không cần cấu hình router
Backend	Node.js, Express, mqtt.js, WebSocket	Nhận dữ liệu, xử lý ngưỡng, cung cấp API cho website, đẩy dữ liệu thời gian thực
Máy chủ	Laptop của nhóm	Chạy backend và phục vụ website khi demo, vấn đáp; khi triển khai thực tế cần máy chủ chạy liên tục
Frontend	HTML, CSS, JavaScript; Chart.js; thư viện gauge	Giao diện đăng nhập, số liệu, biểu đồ, điều khiển, cài đặt
Cơ sở dữ liệu	Firebase (Firestore)	Lưu dữ liệu theo thời gian, tài khoản, ngưỡng cảnh báo
Thông báo	Telegram Bot API	Gửi cảnh báo đến điện thoại người dùng
Quản lý mã nguồn	Git, GitHub	Tích hợp code của cả nhóm trong một repository

6. Chi phí dự kiến

Hạng mục	**Giá dự kiến**
ESP32 DevKit	150.000đ
DHT22	100.000đ
MQ-135	50.000đ
LED RGB WS2812	40.000đ
Buzzer	20.000đ
Adapter nguồn 5 V – 2 A và cáp	40.000đ
Board đồng lỗ, dây, điện trở, transistor; hộp nhựa có lỗ thông khí	80.000đ
Tổng	≈ **480.000đ**

Phần mềm và hạ tầng (HiveMQ Cloud, Firebase, Telegram Bot API) dùng gói miễn phí.